

AI方向学习路线（原CV）

VERSION: 0.0.3

若发现问题或有疑惑，以及修改建议，请联系我们，感谢

Email: moran1024a@gmail.com

本学习路线着重人工智能基础，未来可以转入：大模型、强化学习、模仿学习、具身智能等多个方向。

目录

部分内容没有考核要求，但你不能不会，后面你要是在这些基础上面犯错比你没学完都严重。

- 零、学习原则
- 一、*计算机基本常识和组成（不考核）
- 二、Python 基础
- 三、*Linux 基础（不考核）
- 四、*科学上网（不考核）
- 五、*Git 基础（不考核）
- 六、*AI 工具的使用（不考核）
- 七、神经网络基础

零、学习原则

首先要说明的是，**AI 是一个很广、也很大的方向**。它既有工程的一面，也有学术的一面。你不仅需要工程思维和构建能力，也需要敏捷的思考能力，但更重要的是——**坚持下去的毅力**。

在这份学习路线中，你会看到一些其他地方可能很少强调、甚至“看起来没必要学”的内容。但这些**工具**其实非常重要。掌握它们，你才能真正把自己的想法做出来，而不是被环境、系统和工程问题卡住。

AI 入门之后，你需要逐渐选择一个主要深入的方向。不是因为只能走一条路，而是因为人的时间和精力有限。不同方向之间其实有大量交叉：具身智能可能涉及 CV、模仿学习、强化学习；大模型同样可能涉及强化学习、多模态等内容。

所以，**方向不等于知识边界**。不同方向看起来各不相同，但很多基础其实是相通的。

最后，如果你决定走上这个方向，不要害怕庞大的知识量，也不要畏惧复杂的代码和公式。

会难，但不会学不会。

先学基础，再去实践；遇到问题，再回来补。不断重复这个过程就够了。

最省力学习总则：学完基础的知识之后，在项目中，遇到问题再学，反复迭代，加强。

1. 所有的开发工具，不管是Ubuntu和ros系统，还是c++、python语言，都是工具（而非理论），没有人能完全掌握这些工具，会用就行，能完成项目的开发就算合格；我们对这些工具的掌握在实践中会逐步提高。
2. 学习这些工具的基础之后（速通基础），以项目为驱动，哪里不会再回头学哪里，不要畏难，学习本身就是一个循环反复的过程，很难一遍学会所有东西。
3. 遇到Ubuntu系统终端、程序编译报错，很正常，会有各种各样奇怪的错误；注意看给出的错误信息，然后copy-paste在网上查找，多数问题都能在网上直接找到答案；你也可以直接去问AI，但是要注意判别答案的真伪。
4. 一个人学习遇到困难，可能就放弃了，但是几个人一起学习，就能互相帮助，共同进步；遇到的多数问题都是‘知与不知’的事儿（无关智商），首先自己查找（善于向网络，尤其是语言大模型求助）解决，如果不行，要主动向身边的同学老师求助，把问题解决掉。
5. 好记性不如烂笔头，对于开发同样适用；在学习工具的过程中记录学习笔记（如obsidian，notion，飞书在线笔记等），忘了随时回头看；在开发过程中，每天记录开发笔记，把遇到的问题记录下来。

一、*计算机基本常识和组成（不考核）

需要学什么

- 电脑由哪些主要部分组成
- CPU、GPU、内存、硬盘分别负责什么
- 操作系统是什么，Windows、Linux 等有什么区别
- 文件、文件夹、目录、路径分别是什么
- 绝对路径与相对路径的基本概念
- 常见文件格式和扩展名
- 如何压缩和解压文件
- 如何安装、卸载软件

- 软件版本、安装目录、环境变量等基本概念
- 能看懂一些常见的计算机配置参数

不要求理解 CPU 指令集、操作系统内核、文件系统实现等深入内容。

怎么判断自己学会了

能够简单解释：

- CPU 和 GPU 有什么区别
- 内存和硬盘有什么区别
- 为什么训练 AI 模型通常需要 GPU
- 什么是操作系统
- C:\Users\xxx 或 /home/xxx 表示什么
- .zip、.tar.gz 是什么
- 一个软件下载之后应该如何安装、解压和找到它

遇到“文件在哪里”“软件装在哪里”“为什么找不到这个文件”等问题时，能够自己进行基本排查。

可以看什么视频 / 教材

自己去查资料，这部分我都懒得找资料了，不会可以直接问，但不能不会就不会。

1. b站一个神奇的教程：

<https://www.bilibili.com/video/BV1UwPgziE3F/>

二、Python 基础

Python是人工智能中最常用的编程语言。

需要学什么

重点掌握三个方面：

1. 环境管理（一般是conda，常用miniconda）

- 理解 Python 环境和虚拟环境
- 会创建、切换和管理 Conda 环境
- 会安装和管理 Python 包

2. 三大库

- NumPy：基础数值和数组处理
- Pandas：基础表格和数据处理
- Matplotlib：基础数据可视化

3. 算法

- 洛谷 (<https://www.luogu.com.cn/>) 题单：入门1-6

Python 本身掌握基础语法即可，不需要过早学习复杂的高级特性。

怎么判断自己学会了

能够：

- 独立创建并配置一个 Conda 环境
- 正常安装和使用 Python 包
- 使用 NumPy、Pandas、Matplotlib 完成简单的数据处理和可视化
- 看懂并修改基础 Python 代码
- 具备基本的算法实现能力

可以看什么视频 / 教材

1. 黑马程序员Python课程（其实Python也可以速通，如果你有基础的话甚至可以边用边学）
b站：<https://www.bilibili.com/video/BV1qW4y1a7fU/>
2. 菜鸟教程（有自信就学这个，更快）：
<https://www.runoob.com/python3/python3-tutorial.html>
3. 洛谷 (<https://www.luogu.com.cn/>) 题单：入门1-6
4. 三大库速通课（这个有点过于快了，建议多实操会更扎实）
b站：<https://www.bilibili.com/video/BV1wN4y1T7K9/>

三、*Linux 基础（不考核）

需要学什么

AI 训练服务器、机器人和很多科研环境都会使用 Linux。

这一部分不要求学习 Linux 运维，只需要掌握日常开发中最常用的操作，需要学习以下内容（仅供参考，可以多学一些）。

常用命令（不止这些）：

pwd
ls
cd
mkdir
cp
mv
rm
cat
less
grep
find

了解：

- Linux 的目录结构
- 文件和目录操作
- 文件权限
- sudo
- 用户和管理员权限
- 环境变量
- Shell 的基本概念
- bash / zsh 的基本概念
- 进程是什么

建议多学一点，例如：

- SSH 远程登录
- SCP / rsync 等基本文件传输方式
- 如何查看和结束进程
- 如何运行 Python 程序

怎么判断自己学会了

能够只使用终端完成：

- 创建文件夹
- 进入指定目录
- 复制、移动、删除文件
- 查看文件内容
- 查找一个文件

- 安装并运行一个 Python 项目
- 激活 conda 环境
- 使用 SSH 登录远程服务器
- 将文件传到服务器
- 找到一个正在运行的程序并结束它

不需要背下所有命令，知道遇到问题时应该搜索什么即可。

可以看什么视频 / 教材

自己装一个wsl2玩一玩，但注意wsl2与Linux依然有所差异。

四、*科学上网

需要学什么

AI 这个方向，未来的学习和工作中经常需要访问：

- GitHub
- Hugging Face
- 国外技术文档
- 开源项目主页
- 论文网站
- 模型和数据集下载页面

因此建议了解一些最基本的网络和代理概念。

此部分你需要知道：

- 什么是 IP
- 什么是域名
- 什么是代理
- 什么是 HTTP / HTTPS
- 什么是网络端口
- 系统代理和终端代理的基本区别
- Git、浏览器、Python 等程序可能分别使用自己的网络设置

这一部分以能够正常获取学习资源为目标，不需要深入学习计算机网络。

怎么判断自己学会了

能够：

- 正常访问常见 AI 开发网站
- 下载模型或数据集
- 遇到网络下载失败时，知道可能是网络、代理、地址或工具配置的问题
- 能够进行最基本的排查

可以看什么视频 / 教材

这部分问AI，b站也有一堆科普视频，具体操作不会的就去问学长学姐，有些东西不能光明正大的教。

五、*Git 基础（不考核）

需要学什么

Git 是最常用的代码版本管理工具，GitHub 等代码托管平台基本都基于 Git 使用。

这一部分需要了解：

- Git 和 GitHub 的区别
- 仓库（Repository）是什么
- Commit 是什么
- Branch 是什么
- 本地仓库和远程仓库的区别
- 基本的代码版本管理流程

需要会使用一些常见命令：

```
git clone
git status
git add
git commit
git pull
git push
git branch
git switch
```

不要求学习复杂的 Git 工作流，能够完成日常代码管理即可。

怎么判断自己学会了

能够：

- 从 GitHub 克隆一个项目
- 查看当前仓库状态
- 修改代码后完成一次提交
- 将本地代码推送到远程仓库
- 拉取远程仓库的最新代码
- 创建和切换简单的分支
- 知道 Git 和 GitHub 不是同一个东西
- 遇到简单的 Git 问题时能够自己查找并解决

可以看什么视频 / 教材

Git 不需要专门学得很深，找一个基础教程速通过一遍，然后在实际项目中边用边学即可。

六、*AI 工具的使用

需要学什么

现代 AI 工具可以显著降低编程和学习门槛，但是不能滥用，你需要明确知道**什么时候用有好处，什么时候用没有好处**。

常用的：ClaudeCode、Codex、cc-switch。

重点学习 AI 可以帮助完成什么：

- 解释代码
- 编写简单代码
- 修改代码
- 分析报错
- 阅读项目结构
- 解释论文和概念
- 辅助搜索和整理资料

同时需要了解：

- API 是什么
- API Key 是什么
- 中转站 / API 代理的基本概念
- Token、上下文等基本概念

更重要的是学会**判断 AI 输出是否可靠**。

例如：

- 代码是否真的可以运行
- 命令是否存在
- API 是否已经过时
- AI 是否虚构了参数、函数或库
- AI 给出的结论是否需要查阅官方文档验证

怎么判断自己学会了

能够利用 AI 完成一个简单任务，例如：

1. 把自己的需求说明给 AI
2. 让 AI 给出代码
3. 自己运行代码
4. 根据报错把信息反馈给 AI
5. 修改代码直到运行成功
6. 能够基本判断 AI 的回答是否合理

最重要的判断标准是：

AI 是辅助工具，而不是完全替代自己的理解和判断。

如果能够看懂 AI 给出的主要代码，并知道什么时候需要验证它的答案，就基本达到要求。

可以看什么视频 / 教材

自己b站自己查。

七、神经网络基础

需要学什么

这一部分是正式进入人工智能学习的基础，本文不详细说明怎么学，你可以参考以下的一些视频和教材。

1. #书籍《深度学习入门：基于Python的理论与实现》

这本书是系列丛书的第一本，如果你时间充足可以从这本开始，讲的比较慢比较通俗易懂。

2. #书籍《动手学深度学习：PyTorch版》

这本书也是系列书的其中之一，但是内容更详实，这本书内容较多，不需要学那么多，学前面几章就行了。

3. #视频《PyTorch深度学习快速入门教程（绝对通俗易懂！）【小土堆】》

b站：<https://www.bilibili.com/video/BV1hE411t7RN/>

小土堆老师的课，讲的可以，跟着学没毛病。

4. #视频《吴恩达深度学习入门2024版本的视频》

b站：<https://www.bilibili.com/video/BV17W42197Nm/>

老牌课程，但没那么简单，讲的太多了不推荐入门学习（你要是觉得学起来爽也可以）。

5. #视频《迪哥人工智能课堂》

b站：<https://www.bilibili.com/video/BV1p3pLecEb5/>

讲的比较细，也还可以，就是有点太长了。

怎么判断自己学会了

能回答下面这些问题（仅参考）：

- 神经网络为什么需要训练
- Loss 是什么
- 反向传播在做什么
- 优化器在做什么
- Learning Rate 是什么
- Epoch 和 Batch 是什么
- 训练集和测试集有什么区别
- 什么是过拟合

之后应该能够使用 PyTorch 完成一个最简单的神经网络训练任务。

虽然不要求能够从零推导所有数学公式，但是你不能看不懂，至少要知道什么式子怎么来的代表什么。

如果已经能够看懂一个基础 PyTorch 训练脚本，并知道每一个主要部分在做什么，恭喜你，入门成功。

如果上面的你觉得你已经学完了，直接去拷打学长学姐。